

TEMA-2-POO-COMPLETO.pdf



CamavingaGII



Programación Orientada a Objetos



2º Grado en Ingeniería Informática



**Escuela Superior de Ingeniería
Universidad de Cádiz**



[Accede al documento original](#)

Programación Orientada a Objetos

Versión 1.0/ Tema 2. El paradigma de la POO

CAMAVINGAGII

2025

WUOLAH

ÍNDICE GENERAL

1	Capítulo 1 Introducción	
1.1	Proceso de desarrollo orientado a objetos	4
2	Capítulo 2 Objetos	
2.1	Objetos y propiedades	5
2.2	Tipos de objetos	6
2.3	Mensajes	7
3	Capítulo 3 Clases	
4	Capítulo 4 Relaciones	
4.1	Conceptos previos	9
4.2	Tipos de relaciones	9
5	Capítulo 5 Miembros de clases	
5.1	Estructura básica	11
5.2	Diferencia entre struct y class	12
5.3	Métodos inline	13
5.4	El puntero this	14
5.5	Miembros constantes	16
5.5.1	Atributos constantes	16
5.5.2	Funciones (métodos) constantes	16

5.6	Atributos mutables	18
5.7	Sobrecarga de métodos constantes	19
5.8	Miembros estáticos	20
5.8.1	Atributos de clase	20
5.8.2	Miembros estáticos	21
5.8.3	Consideraciones sobre static	22
5.8.4	Aplicaciones comunes	22
5.9	Sobrecarga de operadores	23
5.9.1	Sobrecarga de + como función externa	24
5.9.2	Sobrecarga de += como miembro	24
5.9.3	Sobrecarga de para flujo de salida	25
5.10	Funciones amigas	26
5.10.1	Casos comunes donde usar friend	26

6 | Capítulo 6

Construcción y uso de objetos

6.1	Constructores y operadores	29
6.1.1	Consideraciones	30
6.1.2	Inicialización uniforme	30
6.1.3	Constructor por defecto o predeterminado	32
6.1.4	Constructor de copia	33
6.1.5	Operador de asignación de copia	36
6.1.6	Constructor de movimiento (C++11)	42
6.1.7	Operador de asignación de movimiento	45
6.1.8	Constructor de conversión	47
6.1.9	Operador de asignación con conversión	49
6.1.10	Constructor de Lista inicializadora (C++11)	52
6.2	Destructor	58
6.2.1	Consideraciones	60

7 | Capítulo 7

Métodos especiales

INTRODUCCIÓN

- En POO un programa se organiza como un conjunto finito de **objetos** que contienen **datos y operaciones** y que se comunican entre sí mediante **mensajes**
- De las **interacciones**, mediante el paso de mensajes entre los objetos que componen un programa, surge la funcionalidad del mismo.

1.1

Proceso de desarrollo orientado a objetos

- Identificar los **objetos** que intervienen
- Agrupar en **clases** los objetos con características y comportamientos comunes.
- Identificar **datos y operaciones** de cada clase
- Identificar las **relaciones** que pueden existir entre las clases
- Los principios en los que se fundamenta la POO son:
 - 1 Abstracción
 - 2 Encapsulamiento
 - 3 Ocultación de información
 - 4 Generalización
 - 5 Polimorfismo

OBJETOS

2.1 Objetos y propiedades

Definición 2.1.1

Un objeto es una entidad individual del problema que se está tratando, formada por la unión de un estado y un comportamiento, que posee un conjunto de datos **atributos** y un conjunto de operaciones **métodos**.

- Cada objeto posee 3 propiedades:
 - 1 **Estado:** Valores de cada atributo del objeto en un instante determinado. Posee un carácter determinado, evoluciona con el tiempo.
 - 2 **Comportamiento:** Qué hace el objeto. Agrupa todas las competencias del objeto y queda definido por las operaciones que posee.
Cuando un objeto **recibe un mensaje activa un método y realiza una acción**.
 - 3 **Identidad:** Permite distinguir un objeto de otro sin ambigüedad. (cada objeto posee una dirección de memoria distinta)
- Los objetos se relacionan con otro mediante enlaces entre objetos. Por ejemplo en la siguiente imagen:
- Podemos observar que el objeto Juan está relacionado mediante un enlace con Cuenta Corriente, y que el objeto Pedro está relacionado con 2 enlaces con Libreta de ahorro y Seguro de vida, respectivamente. Notar que, al estar subrayado hablamos de objetos y no de clases.

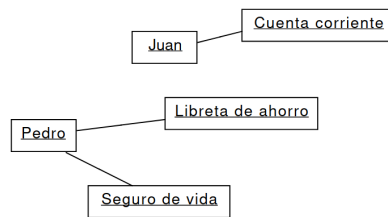


Figura 2.1: Enlaces

2.2

Tipos de objetos

- Los objetos pueden ser de 3 tipos (en cuanto al ámbito de relacionarse con otros objetos se refiere):
 - 1 **Actores:** Objetos que **exclusivamente** emiten mensajes
 - 2 **Servidores:** Objetos que **exclusivamente** reciben mensajes
 - 3 **Agentes:** Objetos que pueden tanto enviar como recibir mensajes.

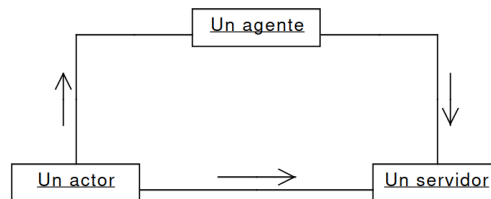


Figura 2.2: Tipos de objetos

2.3

Mensajes

- Hemos dicho que los objetos poseen enlaces unos con otros, pero estos se comunican enviándose **mensajes**.

Un mensaje se compone de:

nombre[destino (receptor), operación (métodos que se deben activar), parámetros(necesarios para que se active el método)]

En la siguiente imagen podemos ver un ejemplo donde el Objeto1 envía un mensaje al Objeto2 (el sentido del mensaje lo indica la punta de flecha). Como resultado de este mensaje el Objeto2 devuelve un dato al Objeto1.

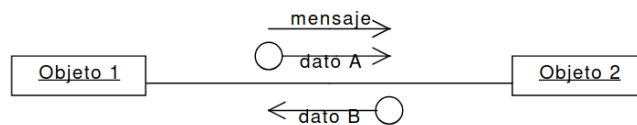


Figura 2.3: Mensajes entre objetos

CLASES

Definición 3.0.1

Una **clase** representa un concepto, idea, entidad... **relevante** del dominio del problema, que a su vez **agrupa un conjunto de objetos** de las mismas características, y es el mecanismo para describir un conjunto de objetos

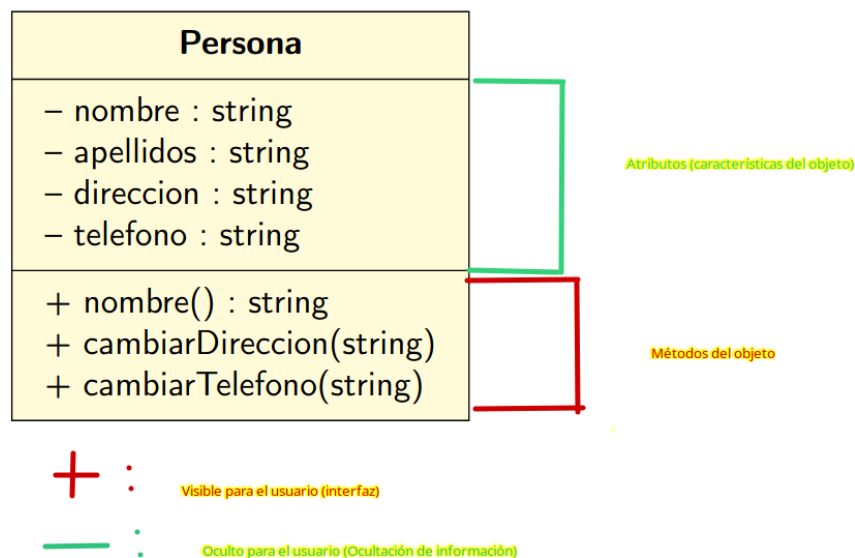


Figura 3.1: Clase

RELACIONES

4.1 Conceptos previos

- **Cardinalidad:** Número de clases relacionadas (binarias, ternarias...)
- **Multiplicidad:** Cuántos objetos intervienen en la relación
- **Navegabilidad:** Quién puede enviar mensajes a quién. Es decir, si ambos pueden enviarse un mensaje, la navegabilidad es bidireccional, si sólo uno de ellos puede enviar un mensaje es unidireccional...

4.2 Tipos de relaciones

- Existen 4 tipos de relaciones entre clases
Nota: Entre clases se denomina **relaciones**, entre objetos se denomina **enlace**.
- **Agregación:** Existen varios tipos:
 - 1 **Asociación por agregación:** Interviene un **agregante** y un **agregado**. Relación entre dos clases donde la **multiplicidad puede variar**. Los objetos de una clase se comunican con los objetos de la otra y viceversa.
En este tipo de relación **no existen restricciones de existencia y multiplicidad**.
Recalcar que en este tipo de relación se pueden enviar mensajes mutuamente (**navegabilidad bidireccional**)
 - 2 **Asociación por composición:** Interviene un **componente** y un **compuesto**. El componente es aquel que forma parte del compuesto. Por ejemplo un libro tiene capítulos, pues los capítulos es el componente del compuesto libro.
Recalcar que un capítulo no puede existir por sí solo, es decir, en la composición, **la existencia del componente por sí sola no tiene sentido sin la existencia del compuesto**.

- **Generalización:** A partir de una clase base, se pueden obtener varias subclases con las mismas propiedades que la clase base y además, con otras distintas que las identifica. Por ejemplo, todo autor es una persona, pero no todas las personas son autores, por tanto, la clase persona es la clase base, y la clase autor es la especialización de la clase persona.
- **Dependencia**
- **Realización**

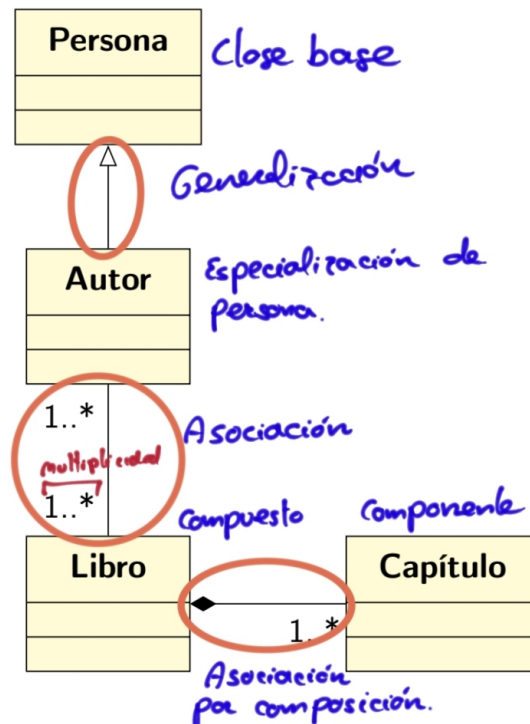


Figura 4.1: Tipos de relaciones

MIEMBROS DE CLASES

5.1

Estructura básica

- Una clase encapsula el conjunto de miembros que la componen:
 - ➊ **Atributos (datos):** Definen la representación de los objetos
 - ➋ **Métodos (funciones):** Definen las operaciones que se pueden realizar con los atributos
 - ➌ **Tipos** auxiliares o relacionados con la clase *typedef, enum*
- Con las palabras reservadas **public** y **private** se controla el acceso a cada uno de los elementos

```
1  class C
2  {
3      public:
4          //Miembros visibles desde el exterior, normalmente metodos y
           tipos, pero no atributos
5
6      private:
7          //Miembros ocultos al exterior (detalles de implementacion de la
           clase), generalmente atributos y a veces metodos auxiliares y
           tipos internos (typedef, enum...)
8  }; //no olvidar este ; al final de la declaracion de la clase
```

5.2

Diferencia entre struct y class

- Las palabras reservadas **struct** y **class** son sinónimas, aunque existen dos diferencias entre ellas, por omisión, los **miembros de class son privados y los de struct son públicos**. La otra diferencia está relacionada con la **herencia**. Por ejemplo, estos 3 códigos son similares:

```
1  class C
2  {
3  //...
4  };
5
6  class C
7  {
8      private:
9      //...
10 };
11
12 struct C
13 {
14 private:
15 //...
16 }
```

- Al igual que este código de aquí es similar:

```
1  struct C
2  {
3  //...
4  }
5
6  struct C
7  {
8      public:
9      //...
10 }
11
12 class C
13 {
14     public:
15     //...
16 }
```

5.3

Métodos inline

Definición 5.3.1

Un método inline es un método (una función de una clase) que se expande en el punto de llamada a dicha función para evitar realizar llamadas al sistema y ganar eficiencia.

- Para declarar un método inline dentro de una clase, tenemos 2 opciones:

```
1
2  class Entero
3  {
4      public:
5          int cambiar(int x) //al desarrollar la propia funcion en el
6                               momento de ser declarada el compilador lo toma como si
7                               fuese un metodo inline
8          {
9              int ant=i;
10             i=x;
11             return ant;
12         }
13     private:
14         int i;
15 }
16
17 //Otra forma de implementarlo
18
19 class Entero
20 {
21     public:
22         int cambiar(int x);
23     private:
24         int i;
25 }
26
27 inline int Entero::cambiar(int x) //se declara en el mismo .h donde
28                                     esta declarada la clase
29 {
30     int ant=i;
31     i=x;
32     return ant;
33 }
```

5.4

El puntero this

Definición 5.4.1

El puntero this es el primer parámetro implícito de los métodos de una clase. This es de tipo X*(puntero a X) y no es posible cambiarlo para que apunte a otro objeto. Este puntero sirve para referenciar al objeto con el que estamos trabajando. NOTA: Dentro de un método, para referirnos al objeto con el que estamos trabajando con este puntero, lo hacemos con *this.atributo o this->atributo. Un ejemplo:

```
1  class X
2  {
3      int m,n; //por defecto variables privadas
4      public:
5          void fun (int n) //esto el compilador lo traduce a: void fun
6                          (X* const this). Puntero constante a X. Este puntero no se
7                          puede modificar para que apunte a otro objeto
8          {
9              if(n%2)
10                 m=n;
11                 this->n=n; //como el parametro de la funcion se llama
12                         igual que el atributo de la clase hemos de usar el
13                         puntero this.
14             }
15     };
16
17 //Si llamasemos al metodo desde el main por ejemplo:
18 X numero;
19 numero.fun(7); // el compilador interpreta: X::fun(&numero,7);
```

- El puntero this se utiliza en las siguientes situaciones (sacado de ChatGPT porque Josefi no ha explicado cuándo se usa):

- 1 **Ambigüedad de nombres:** Cuando los parámetros de un método tienen el mismo nombre que las variables miembro de la clase, el puntero this se usa para diferenciar entre ellos.
- 2 **Retorno en cadena:** Se puede usar this para devolver un puntero o una referencia al objeto actual, lo que permite encadenar llamadas a métodos. Por ejemplo:

```
1 class Calculadora {
2 private:
3     int valor;
4
5 public:
6     Calculadora& sumar(int x) {
7         valor += x;
8         return *this; // Se devuelve el objeto actual
9     }
10
11     Calculadora& multiplicar(int x) {
12         valor *= x;
13         return *this; // Encadenar otro metodo
14     }
15
16     int obtenerValor() const {
17         return valor;
18     }
19 };
20
21 int main() {
22     Calculadora calc;
23     int resultado = calc.sumar(10).multiplicar(2).obtenerValor(); //
24     // Llamadas encadenadas
25     std::cout << "Resultado: " << resultado << std::endl;
26     return 0;
27 }
```

- **Comparación de objetos:** Se puede usar this para comparar el objeto actual con otro objeto. (este punto sería el tercero, pero al insertar código de por medio se me ha ido de nuevo al punto 1.) Por ejemplo:

```
1 class Persona {
2 private:
3     int edad;
4
5 public:
6     Persona(int edad) : edad(edad) {}
7
8     bool esIgualA(const Persona& otra) const {
9         return this->edad == otra.edad; // Comparar con otro objeto
10    }
11};
```


Definición 5.5.1

Los miembros constantes de una clase son aquellos atributos y/o funciones que no modifican el estado del objeto. Es decir:

5.5.1. Atributos constantes

- El valor de un atributo declarado **const** no puede ser modificado después de ser inicializado (normalmente por el constructor)
- Los atributos de un objeto constante se tratan como si todos sus atributos fuesen constantes, exceptuando los que se han declarado **mutable**.

5.5.2. Funciones (métodos) constantes

- Son **métodos observadores** aquellos que informan del estado del objeto **sin modificarlo** y se distinguen con la palabra **const** que se pone después de los parámetros.
Dicho **const** **se aplica al objeto apuntado por el parámetro this**, por lo que ahora el puntero **this** **NO será un puntero constante a un objeto**, si no será **un puntero constante a un objeto constante**, o lo que es lo mismo, un puntero que no puede apuntar a otro objeto, cuyo objeto al que apunta no puede ser modificado.
Es decir, **const X* const this**.
- Obviamente, un método **const** **no puede modificar el estado del objeto** a menos que modifique los atributos declarados como **mutable**
- Un ejemplo:

```

1
2  class Reloj
3  {
4  public:
5      //...
6      void incrementarHoras() //void incrementarHoras(Reloj* const this
7          )
8          {h_=(h+1) %24;}
9
10     void incrementarMinutos() //void incrementarMinutos(Reloj* const
11         this)
12     {
13         if((m_=(m_+1) %60)==0)
14             incrementarHoras();
15     }
16
17     int horas() const //void horas(const Reloj* const this). Miembro
18         constante
19     {return h_;}
20
21     int minutos() const //void minutos(const Reloj* const this).
22         Miembro constante
23     {return m_;}
24
25 private:
26     int m_,h_; //minutos y horas
27 }

```

Definición 5.6.1

Un atributo **mutable** es aquel que siempre se puede modificar, aún cuando el objeto al que pertenezca sea constante. Un atributo mutable no afecta al estado de los objetos visible desde el exterior.

- Un atributo mutable se usa cuando necesitamos modificarlo en métodos const, es decir, en métodos que no deberían modificar el objeto, pero aún así queremos cambiar ese atributo.
- Debemos tener en cuenta 2 estados de un objeto con este tipo de atributo:
 - 1 **Estado lógico:** Es el estado del objeto sin el atributo mutable, es decir, el valor de los atributos del objeto sin tener en cuenta el atributo mutable.
 - 2 **Estado físico:** Es el estado del objeto teniendo en cuenta el atributo mutable. Veamos un ejemplo:

```
1  class Semaforo
2  {
3      enum Color{ROJO, VERDE, AMBAR};
4
5      public:
6          void comprobar() const{revisado=true;} //no cambia el estado
7              logico del objeto
8          void cambiar(){c=((c+1)%3);}
9
10         private:
11             Color c;
12             mutable bool revisado; //no afecta al estado logico del objeto (
13                 el color)
```

5.7

Sobrecarga de métodos constantes

- Un objeto **const** no puede llamar a un método **no-const** (es obvio, si todos los atributos de un objeto son constantes o **this** es un puntero a un objeto constante que no se puede modificar, como va a llamar a una función que **puede** modificar esos atributos), porque un objeto **const** de tipo **X** no puede pasarse como parámetro implícito de tipo **X*** (puntero a un **X** no-const).
- En el siguiente ejemplo se muestra como un metodo const intenta llamar a un metodo no const, dando error. Aunque este metodo no const no modifique nada, el puntero **this** es un puntero a un objeto constante, lo que en los otros metodos no const no lo es:

```
1 class Alumno{
2     public:
3         //...
4         string& nombre(){return nombre_;}
5         unsigned long& dni(){return dni_;}
6         void mostrar() const{
7             cout<<"Nombre:" <<nombre()<<endl;
8             cout<<"DNI:"<<dni()<<endl; //ERROR en ambos cout, ya que una
              funcion const posee el puntero this a un objeto constante, es
              decir: const *Alumno const this, mientras que en los metodos
              nombre y dni no.
9         }
10
11     private:
12         string nombre_;
13         unsigned long dni_;
14
15 };
```

Definición 5.8.1

El modificador `static` en C++ indica que un miembro (atributo o método) pertenece a la clase en lugar de a los objetos individuales. Esto tiene implicaciones clave tanto en cómo se accede a estos miembros como en su comportamiento en memoria.

5.8.1. Atributos de clase

- Un atributo de clase representa una propiedad **de la clase, no de objetos individuales**, y se declara con la palabra reservada **`static`**. A diferencia de un atributo normal, un atributo estático es compartido **por todos los objetos de la clase**.
- Se declaran dentro de la clase con `static`, pero deben definirse fuera para reservar memoria.
- Son ideales para valores o contadores que deban mantenerse globales para todas las instancias.
- No se accede a ellos a través del parámetro `this`.
- Un ejemplo:

```
1 class Contador {
2     static int total; // Declaracion del atributo estatico
3 public:
4     Contador() { total++; }
5     ~Contador() { total--; }
6     static int obtenerTotal() { return total; } // Metodo estatico para
7         consultar el total
8 };
9
10 // Definicion del atributo estatico
11 int Contador::total = 0;
12
13 int main() {
14     Contador c1, c2;
15     std::cout << "Total: " << Contador::obtenerTotal() << "\n"; // Salida
16         : Total: 2
17     return 0;
18 }
```

5.8.2. Miembros estáticos

- Un método estático, declarado también con la palabra `static`, es una función auxiliar de una clase que **no necesita operar con objetos individuales**, por esta razón, **no recibe el parámetro implícito `this`** y no tiene acceso a los miembros no estáticos. En definitiva:
 - No reciben el parámetro implícito `this`.
 - Solo pueden usar otros miembros estáticos (atributos o métodos).
 - Se utilizan a menudo para utilidades generales o cálculos relacionados con la clase.
- Por ejemplo:

```
1 class Matematica {
2 public:
3     static int suma(int a, int b) { return a + b; } // Metodo estatico
4 };
5
6 int main() {
7     std::cout << "Suma: " << Matematica::suma(5, 7) << "\n"; // Llamada
8         directa usando el nombre de la clase
9     return 0;
10 }
```

- Existen 2 formas de acceder a los miembros estáticos:
 - 1 Operador de resolución de ámbito (**X::m**): Es la forma recomendada, ya que indica claramente que se está accediendo a un miembro estático de la clase.
 - 2 A través de objetos (**E.m o E->m**): Aunque posible, no se recomienda porque oculta la naturaleza estática del miembro.
- Un ejemplo:

```
1 class Ejemplo {
2 public:
3     static int valor;
4 };
5
6 int Ejemplo::valor = 10;
7
8 int main() {
9     Ejemplo e;
10    std::cout << Ejemplo::valor << "\n"; // Recomendado
11    std::cout << e.valor << "\n";       // Permitido, pero no ideal
12    return 0;
13 }
```

5.8.3. Consideraciones sobre static

- **Atributos estáticos en clases derivadas:** Los atributos estáticos se comparten en toda la jerarquía de clases. No se duplican en las clases derivadas.
- **Duración de vida:** Los miembros estáticos existen desde el inicio hasta el final del programa, independientemente del número de objetos creados.
- **Incompatibilidad con virtual:** Un método static no puede ser virtual, ya que no opera sobre objetos individuales.
- **Incorporación en plantillas (template):** Los miembros estáticos en clases plantillas son independientes para cada especialización.
- Los atributos static const de tipos enteros se pueden definir dentro de la clase, los demás, se deben definir fuera.

5.8.4. Aplicaciones comunes

- 1 **Contadores globales:** Para llevar un registro de cuántos objetos de una clase se han creado o eliminado.
- 2 **Funciones auxiliares:** Métodos que no necesitan interactuar con atributos específicos del objeto.
- 3 **Singletons:** Uso de atributos estáticos para almacenar la única instancia de una clase. Un ejemplo de una clase Singleton:

```
1 class Singleton {
2     static Singleton* instancia; // Atributo estatico que almacena la
        instancia unica
3     Singleton() {} // Constructor privado
4 public:
5     static Singleton* obtenerInstancia() {
6         if (!instancia)
7             instancia = new Singleton();
8         return instancia;
9     }
10 };
11
12 // Inicializacion del atributo estatico
13 Singleton* Singleton::instancia = nullptr;
14
15 int main() {
16     Singleton* s1 = Singleton::obtenerInstancia();
17     Singleton* s2 = Singleton::obtenerInstancia();
18     std::cout << (s1 == s2) << "\n";
19     // Salida: 1 (ambos apuntan al mismo objeto)
20     return 0;
21 }
```

Definición 5.9.1

Sobrecargar un operador significa tomar un operador que ya existe, como por ejemplo el operador de suma +, y reimplementar el operador pertinente para que opere con objetos de tu clase.

- La mayoría de los operadores de C++ pueden ser sobrecargados para que al menos uno de sus operandos sea de un tipo definido por el programador.
- **No se puede cambiar la sintaxis (número y orden de operandos)**, la precedencia ni la asociación de un operador. Tampoco es posible crear nuevos operadores.
- **No se debe** cambiar la semántica de los operadores o ser incoherente definiendo los relacionados, por ejemplo, ++x es lo mismo que x+=1 y x=x+1. Esas 3 operaciones deben ser equivalentes
- Hay operadores que **no se pueden sobrecargar** como el operador de ámbito ::, el de selección de miembro ., el de selección de miembro a través de un puntero .*, evaluación condicional ?:, tamaño de un tipo u objeto sizeof...
- Salvo excepciones, los operadores que se pueden sobrecargar son:
 - Miembros no estáticos de la clase. Estos reciben un parámetro menos, ya que el primer parámetro es el implícito this.
 - Funciones que no pertenecen a ninguna clase. Todos sus operandos se declaran como parámetros
- Las reglas gramaticales de C++ obligan a que sean **miembros no estáticos** los siguientes operadores = (asignación), () (llamada a función), [] (indexación) y -> (selección de miembro a través de un puntero)
- Ciertos operadores deben ser **no miembros** porque el tipo de su primer operando no es una clase o, si lo es, no se le pueden añadir nuevos miembros, como es el caso de « (inserción en flujo) y » (extracción de flujo).
- **Los operadores que modifican el estado de su primer operando es preferible que sean miembros** (p.e. *= y ++), ya que su implementación suele ser más eficiente accediendo a la representación interna de la clase
- En cambio, **si se necesitan conversiones implícitas** en ambos operandos (p. ej., +, ==), es decir, **convertir un operador que no es un objeto de la clase a un objeto de la clase**, la declaramos como función externa para tener flexibilidad para operar con diferentes tipos.
- Veamos varios ejemplos:

5.9.1. Sobrecarga de + como función externa

```
1 #include <iostream>
2
3 class Vector {
4     double x, y;
5 public:
6     Vector(double x = 0, double y = 0) : x(x), y(y) {}
7
8     // Acceso publico para mostrar resultados.
9     void print() const { std::cout << "(" << x << ", " << y << ")\n"; }
10
11     // Declaracion de friend para acceso privado.
12     friend Vector operator+(const Vector& v1, const Vector& v2);
13 };
14
15 Vector operator+(const Vector& v1, const Vector& v2) {
16     return Vector(v1.x + v2.x, v1.y + v2.y);
17 }
18
19 int main() {
20     Vector v1(1, 2), v2(3, 4);
21     Vector v3 = v1 + v2;
22     v3.print(); // Salida: (4, 6)
23 }
```

5.9.2. Sobrecarga de += como miembro

```
1 class Vector {
2     double x, y;
3 public:
4     Vector(double x = 0, double y = 0) : x(x), y(y) {}
5
6     // Sobrecarga de += como miembro
7     Vector& operator+=(const Vector& v) {
8         x += v.x;
9         y += v.y;
10        return *this;
11    }
12
13    void print() const { std::cout << "(" << x << ", " << y << ")\n"; }
14 };
15
16 int main() {
17     Vector v1(1, 2), v2(3, 4);
18     v1 += v2;
19     v1.print(); // Salida: (4, 6)
20 }
```

5.9.3. Sobrecarga de para flujo de salida

```
1 #include <iostream>
2 class Vector {
3     double x, y;
4 public:
5     Vector(double x = 0, double y = 0) : x(x), y(y) {}
6
7     friend std::ostream& operator<<(std::ostream& os, const Vector& v);
8 };
9
10 std::ostream& operator<<(std::ostream& os, const Vector& v) {
11     return os << "(" << v.x << ", " << v.y << ")";
12 }
13
14 int main() {
15     Vector v(1, 2);
16     std::cout << v << "\n"; // Salida: (1, 2)
17 }
```

5.10

Funciones amigas

Definición 5.10.1

Una función amiga de una clase es una función que no es miembro de ella pero tiene acceso a su implementación privada al igual que las funciones miembro.

5.10.1. Casos comunes donde usar friend

- **Operadores sobrecargados:** Cuando se sobrecargan operadores binarios como +, -, <, >, etc., es común que se declaren como friend para acceder directamente a los datos privados. Por ejemplo:

```
1 #include <iostream>
2 class Punto {
3     int x, y;
4 public:
5     Punto(int x_val, int y_val) : x(x_val), y(y_val) {}
6
7     // Declarar friend para permitir acceso a miembros privados
8     friend std::ostream& operator<<(std::ostream& os, const Punto& p);
9 };
10
11 // Sobrecarga del operador << para imprimir Punto
12 std::ostream& operator<<(std::ostream& os, const Punto& p) {
13     os << "(" << p.x << ", " << p.y << ")";
14     return os;
15 }
16
17 int main() {
18     Punto p(3, 4);
19     std::cout << p << std::endl; // Salida: (3, 4)
20 }
```

- **Funciones auxiliares que necesitan acceso a datos privados:** Si una función no es parte de la clase, pero necesita manipular sus datos privados, se puede declarar como friend. Por ejemplo:

```
1 class Caja {
2     int ancho, alto;
3 public:
4     Caja(int a, int h) : ancho(a), alto(h) {}
5
6     // Funcion amiga
7     friend int calcularArea(const Caja& c);
8 };
9
10 // Implementacion de la funcion amiga
11 int calcularArea(const Caja& c) {
12     return c.ancho * c.alto;
13 }
14
15 int main() {
16     Caja c(5, 10);
17     std::cout << "Area: " << calcularArea(c) << std::endl; // Salida:
18     Area: 50
19 }
```

- **Relaciones entre clases (amistad entre clases):** Una clase entera puede ser declarada friend para que acceda a los miembros privados/protegidos de otra clase.

```
1 class B;
2 class A {
3 private:
4     int dato;
5 public:
6     A(int valor) : dato(valor) {}
7     friend class B; // Toda la clase B puede acceder a los miembros
                       // privados de A
8 };
9
10 class B {
11 public:
12     void mostrarDatoDeA(const A& a) {
13         std::cout << "Dato de A: " << a.dato << std::endl;
14     }
15 };
16
17 int main() {
18     A a(42);
19     B b;
20     b.mostrarDatoDeA(a); // Salida: Dato de A: 42
21 }
```

CONSTRUCCIÓN Y USO DE OBJETOS

6.1

Constructores y operadores

Definición 6.1.1

Un **constructor** no es más que una función perteneciente a una clase cuyo objetivo es **crear e inicializar un objeto**.

Una clase puede tener más de un constructor, pero siempre tiene que tener al menos uno.

- Existen distintos tipos de constructores, los cuáles podemos observar en la siguiente tabla:

Método	Parámetros		Descripción
	Número	Tipo	
Constructor	> 1	cualquiera	Genera un objeto nuevo.
Constructor predeterminado	0		Objeto en estado inicial por defecto.
Constructor de conversión	1	$\neq$ clase	Convierte el parámetro en un nuevo objeto.
Constructor de copia	1	clase &	Genera un clon sin modificar el original.
Constructor de movimiento	1	clase &&	Genera un clon «vacian-do» el original.
Constructor de lista inicializadora	1	initializer_list<T>	Construye un contenedor con los elementos de la lista.

Figura 6.1: Tipos de constructores

6.1.1. Consideraciones

- Los constructores **NO** se llaman manualmente, se llaman automáticamente por el compilador cuando:
 - Cuando se define un nuevo objeto
 - Al crear un objeto con memoria dinámica: **new y new[]**
La diferencia entre new y new[] es que ambos se utilizan para reservar memoria devolviendo un puntero a dicha zona de memoria reservada, pero new crea el objeto llamando al constructor y new[] crea el objeto llamando al constructor predeterminado.
 - En la transferencia por valor de parámetros y resultados de funciones.
 - Cuando se requiere una conversión de un parámetro de un tipo que no es el de la clase, utilizando el constructor de conversión.
- Un constructor **NO** devuelve nada, ni siquiera void y el nombre de un constructor es idéntico con el de su clase.
- Pueden poseer una **lista de inicialización**, que como su propio nombre indica, se utiliza para inicializar los atributos. Es muy necesario utilizarlo cuando, por ejemplo, un objeto va a poseer atributos constantes que posteriormente no van a poder ser modificados.
- Si no se inicializa cada atributo en la lista de inicialización, cada uno tendrá un valor indeterminado (basura), lo que hace que si, por ejemplo, tengamos atributos constantes, no podamos modificar su contenido, por lo que siempre es conveniente inicializar los atributos en dicha lista.

6.1.2. Inicialización uniforme

- Desde C++11 se pueden usar varios estilos de inicialización, pero nosotros utilizaremos aquel que utiliza las llaves, ya que con este método de inicialización, evitamos que cualquier atributo se quede con un valor basura, ya que entre otras propiedades que tiene, **asegura la inicialización por defecto incluso de tipos básicos**. Veamos sus propiedades:
 - ❶ Es **uniforme**, es decir, siempre tiene el mismo significado: crear un valor u objeto del tipo declarado determinado por el inicializador.
 - ❷ Es **universal**. Se puede usar en cualquier contexto en el que haya un nuevo objeto o inicializar una variable.
 - ❸ **Impide conversiones peligrosas** con truncamiento o estrechamiento.
 - ❹ Asegura la inicialización por defecto incluso de tipos básicos
 - ❺ Un ejemplo:

```

1 #include <iostream>
2 #include <string>
3
4 class Persona {
5 public:
6     std::string nombre;
7     int edad;
8
9     // Constructor
10    Persona(std::string n, int e) : nombre(n), edad(e) {} //lista de
        inicializacion
11 };
12
13 int main() {
14     // Inicializacion con llaves {}
15     Persona p1{"Maria", 30};
16
17     std::cout << "Nombre: " << p1.nombre << ", Edad: " << p1.edad << std
        ::endl;
18
19     return 0;
20
21
22
23 /*
24     EJEMPLOS:
25     int y{10.5}; //ERROR. Las llaves evitan convertir un entero a un
        double
26     float w{8.35}; //ERROR. Conversion de estrechamiento, un float tiene
        menos precision que un double
27 */
28 }

```


6.1.3. Constructor por defecto o predeterminado

- El constructor por defecto es aquel que **puede ser llamado sin parámetros**. Se encarga de inicializar con valores predeterminados a los atributos.
- Si no definimos ningún constructor para una clase, el compilador genera un constructor predeterminado por defecto, que no hace nada, es decir, como si hiciésemos un constructor vacío, dejando con valores indeterminados a los atributos. Por ejemplo:

```
1
2 class Punto
3 {
4     public:
5     Punto(double x, double y): x_{x}, y_{y} //lista de inicializacion
6     {}
7     //...
8
9     private:
10    double x_, y_;
11 };
12
13 // en una parte donde se utilice la clase Punto ...
14 Punto a{3., 4.}; // Bien, se llama a a.Punto(3.0, 4.0)
15 Punto* p = new Punto{0., 0.}; // Bien, en memoria dinamica
16 Punto b; // ERROR, no hay ctor. predeterminado
17 Punto* pp = new Punto; // ERROR, no hay ctor. predeterminado
18 Punto* vp = new Punto[5]; // ERROR, no hay ctor. predeterminado
19 Punto c[5]; // ERROR, no hay ctor. predeterminado
```

- Ahora, un ejemplo donde lo anterior sería válido:

```
1 class Punto {
2 public:
3     Punto() : x_{}, y_{} {} // constructor predeterminado
4     Punto(double x, double y) : x_{x}, y_{y} {} // constructor
5     // ...
6 private:
7     double x_, y_;
8 };
9
10 //Con esta declaracion de la clase el programa anterior es valido
```

6.1.4. Constructor de copia

- El constructor de copia se usa para crear un nuevo objeto como una copia exacta de otro objeto del mismo tipo. Se invoca cuando se inicializa un objeto a partir de otro existente.
- Se le pasa el objeto **por referencia** ya que si fuese por valor se producirían **llamadas recursivas infinitas**, ya que en ese parámetro se tendría que copiar también los atributos del objeto en cuestión.
- Se pasa por referencia constante para protegernos de modificar el objeto.
- En definitiva, la sintaxis es: Clase(const Clase& otro);
- Un ejemplo:

```
1 #include <iostream>
2
3 class Persona {
4 public:
5     std::string nombre;
6     int edad;
7
8     // Constructor normal
9     Persona(std::string n, int e) : nombre(n), edad(e) {}
10
11    // Constructor de copia
12    Persona(const Persona& otra) {
13        nombre = otra.nombre;
14        edad = otra.edad;
15        std::cout << "Constructor de copia llamado" << std::endl;
16    }
17
18    void mostrar() {
19        std::cout << "Nombre: " << nombre << ", Edad: " << edad << std::
20            endl;
21    }
22 };
23
24 int main() {
25     Persona p1("Juan", 25);
26     Persona p2 = p1; // Llama al constructor de copia
27
28     p2.mostrar(); // Imprime: Nombre: Juan, Edad: 25
29
30     return 0;
31 }
32
33 /*
34 ----- SALIDA ESPERADA -----
35 Constructor de copia llamado
36 Nombre: Juan, Edad: 25
37 */
```

¿Cuándo se invoca al constructor de copia?

- Cuando se inicializa un objeto a partir de otro del mismo tipo:
Persona p2 = p1; // Llama al constructor de copia
- Cuando se pasa un objeto por valor a una función:
void funcion(Persona p); // Se hace una copia del objeto
- Cuando una función devuelve un objeto por valor:

```
1 Persona crearPersona() {  
2     Persona p("Carlos", 30);  
3     return p; // Llama al constructor de copia  
4 }
```

Problema del constructor de copia por defecto

- Si una clase usa punteros para manejar memoria dinámica, el constructor de copia por defecto hace una copia superficial, lo que puede causar problemas de doble liberación de memoria.

```
1 #include <iostream>  
2  
3 class Dato {  
4 public:  
5     int* valor;  
6  
7     Dato(int v) { //constructor  
8         valor = new int(v); // Reserva memoria dinamica  
9     }  
10  
11     ~Dato() {  
12         delete valor; // Libera la memoria  
13     }  
14 };  
15  
16 int main() {  
17     Dato d1(10);  
18     Dato d2 = d1; // Copia por defecto (copia superficial)  
19  
20     // Cuando d1 y d2 se destruyen, ambos intentan liberar la misma  
21     // memoria.  
22     return 0;  
23 }  
24  
25  
26 // Esto causa un comportamiento indefinido, ya que d1 y d2 comparten el  
    // mismo puntero valor.
```

Solución: Implementar un Constructor de Copia Profunda

```
1 #include <iostream>
2
3 class Dato {
4 public:
5     int* valor;
6
7     // Constructor
8     Dato(int v) {
9         valor = new int(v);
10    }
11
12    // Constructor de copia (copia profunda)
13    Dato(const Dato& otro) {
14        valor = new int(*otro.valor); // Crea una nueva copia de la
15                                       memoria
16    }
17
18    ~Dato() {
19        delete valor; // Libera la memoria
20    }
21
22    void mostrar() {
23        std::cout << "Valor: " << *valor << std::endl;
24    }
25 };
26
27 int main() {
28     Dato d1(10);
29     Dato d2 = d1; // Llama al constructor de copia (copia profunda)
30
31     d2.mostrar(); // Imprime: Valor: 10
32
33     return 0;
34 }
35
36 // Ahora d1 y d2 tienen copias independientes en memoria, evitando el
37 // problema de doble liberación.
```

6.1.5. Operador de asignación de copia

Definición 6.1.2

El operador de asignación de copia (**operator=**) es un operador sobrecargable que se usa para asignar los valores de un objeto de una clase a otro objeto de la misma clase después de que ambos hayan sido creados.

El operador de asignación de copia recibe un objeto de la clase (operando derecho) para copiarlo en otro preexistente (operando izquierdo) que devuelve por referencia. Su sintaxis general es la siguiente:

```
1 Clase& Clase::operator=(const Clase& otro) {  
2     if (this != &otro) { // Evitar autoasignación  
3         // Copiar los miembros de 'otro' en 'this'  
4     }  
5     return *this; // Devolver la referencia al objeto actual  
6 }
```

- Este operador **tiene que ser miembro de la clase**, no una función friend. Es definido implícitamente por el compilador si el programador no lo define.
- **El operador de asignación de copia por defecto** asigna uno a uno los atributos de la clase, usando el operador de asignación de cada atributo.
- **NOTA:** Devolvemos `*this` para operaciones encadenadas, por ejemplo: `x=y=z=a=b`. Devolvemos el resultado por referencia para evitar copias innecesarias.
- En el siguiente ejemplo, no se definirá un operador de asignación de copia, y provocará un error de ejecución. Veamos el código:

memoria, por lo que cuando termina la función *ver_datos*, lo que hace es llamar al destructor y liberar dicha zona de memoria, quedándose el objeto pp del main sin un nombre, ya que esa zona de memoria acaba de ser liberada.

- Por tanto, la solución sería definir un constructor de copia donde reservemos **nueva memoria** cuando se vaya a copiar un objeto:

```
1 // Clase Persona, version con copia definida por el programador
2 #include <cstring> // strlen, strcpy, strcmp
3 class Persona {
4     char* nombre_;
5     int edad_, estatura_;
6 public:
7     Persona(const char* n, int ed, int est) :
8         nombre_{new char[std::strlen(n) + 1]},
9         edad_{ed}, estatura_{est}
10    {
11        std::strcpy(nombre_, n);
12    }
13    Persona(const Persona& otra) : // Delegacion de constructor (C++11),
14        // evitamos la repeticion de codigo, ya que se hace exactamente lo
15        // mismo que en el constructor. Se ha de llamar en la lista de
16        // inicializacion
17    Persona{otra.nombre_, otra.edad_, otra.estatura_} {}
18    Persona& operator =(const Persona& otra);
19    ~Persona() { delete[] nombre_; }
20    void estatura(int e) { estatura_ = e; }
21    int estatura() const { return estatura_; }
22    void edad(int e) { edad_ = e; }
23    int edad() const { return edad_; }
24    char* nombre() const { return nombre_; }
25 };
```

- Ahora, antes de implementar la operación de asignación por copia, hemos de tener en cuenta que **debemos manejar el caso donde el programador haga a=a**. (por ello ponemos la condición de que `this!=&otro`).
- Por otro lado, hemos de tener en cuenta que **primero debemos realizar las tareas problemáticas**, que son aquellas que piden recursos al sistema y cabe la posibilidad de que el sistema **NO** otorgue los recursos pedidos.
Por ello, si lo realizamos primero y esta petición falla, el operador *new* en este caso lanzará una excepción cortándose la ejecución de la función, **dejando el objeto intacto** a como estaba anteriormente.
- En este caso, si **las cadenas tienen la misma longitud, no haría falta reservar memoria, si no que directamente machacamos el contenido reemplazando carácter a carácter de la cadena antigua por la nueva**. Por tanto, una posible implementación podría ser:

```

1 // ----- OPERADOR DE ASIGNACION POR COPIA -----
2
3 Persona& Persona::operator =(const Persona& otra)
4 {
5     if(std::strlen(nombre_)==std::strlen(otra.nombre_)) //misma longitud,
6         copiamos
7     {
8         std::strcpy(nombre_,otra.nombre_); //copiamos la cadena
9         edad_otra.edad_;
10        estatura_=otra.estatura_;
11    }
12
13    else
14    {
15        Persona p{otra}; //delegacion de constructor de nuevo. Creamos un
16                           objeto p de la clase persona que contendrá una copia de los
17                           atributos del objeto otra (se llamaría al constructor de copia
18                           )
19        std::swap(nombre_,p.nombre_);
20        std::swap(edad_,p.edad_);
21        std::swap(estatura_,p.estatura_);
22
23        //luego de estos intercambios, el objeto p se quedaria con los
24        //antiguos valores que poseia el objeto this. Al salir de esta
25        //funcion p sera destruido
26    }
27
28    return *this;
29 }

```

- El main sería el siguiente:

```

1
2 /* ----- PROGRAMA PRINCIPAL (MAIN)-----*/
3 #include <iostream>
4 void ver_datos(Persona p)
5 {
6     std::cout << p.nombre() << " tiene " << p.edad() << " años "
7     << "y mide " << p.estatura() / 100.0 << " m.\n";
8 }
9 int main()
10 {
11     Persona pp{"Pepe", 32, 180};
12     ver_datos(pp);
13     std::cout << pp.nombre()
14     << (std::strcmp(pp.nombre(), "Pepe") ? "" : " NO")
15     << " ha sido destruido!\n\n";
16 }
17
18 /*----- SALIDA: -----
19 Pepe tiene 32 años y mide 1.8 m.

```



```
20     Pepe NO ha sido destruido!  
21     */
```

- Otro ejemplo:

```
1 #include <iostream>  
2 #include <cstring>  
3 #include <utility> // Para std::swap  
4  
5 class Libro {  
6 private:  
7     char* titulo;  
8     int paginas;  
9  
10 public:  
11     // Constructor delegante  
12     Libro(const char* titulo = "", int paginas = 0)  
13         : titulo(new char[strlen(titulo) + 1], paginas(paginas) {  
14         strcpy(this->titulo, titulo);  
15     }  
16  
17     // Constructor de copia  
18     Libro(const Libro& otro)  
19         : Libro(otro.titulo, otro.paginas) {} // Delegacion de  
20         constructor  
21  
22     // Operador de asignacion usando copy-and-swap  
23     Libro& operator=(Libro otro) { // Nota: el argumento se pasa por  
24         valor (copia implicita)  
25         swap(*this, otro); // Intercambia datos con el objeto temporal  
26         return *this;  
27     }  
28  
29     // Destructor  
30     ~Libro() {  
31         delete[] titulo;  
32     }  
33  
34     // Metodo para mostrar la informacion del libro  
35     void mostrar() const {  
36         std::cout << "T tulo: " << titulo << ", P ginas: " << paginas  
37         << std::endl;  
38     }  
39  
40     // Funcion amiga swap para intercambiar datos  
41     friend void swap(Libro& primero, Libro& segundo) noexcept {  
42         std::swap(primero.titulo, segundo.titulo);  
43         std::swap(primero.paginas, segundo.paginas);  
44     }  
45 };  
46  
47 int main() {  
48     Libro libro1("C++ Avanzado", 500);
```

```
46     Libro libro2("Fundamentos de Programacion", 300);
47
48     libro2 = libro1; // Se usa el operador de asignacion de copia
49
50     libro1.mostrar();
51     libro2.mostrar();
52
53     return 0;
54 }
55
56 /* -----SALIDA-----
57 Titulo: C++ Avanzado, Paginas: 500
58 Titulo: C++ Avanzado, Paginas: 500
59 */
```

6.1.6. Constructor de movimiento (C++11)

Definición 6.1.3

El constructor de movimiento es un tipo especial de constructor que se usa para mover recursos en lugar de copiarlos.

El constructor de movimiento **se llama cuando un objeto se inicializa a partir de un objeto temporal** (rvalue). En lugar de hacer una copia completa, **transfiere los recursos del objeto fuente al nuevo objeto** y deja el **objeto original** en un estado válido pero **vacío**.

Este constructor lo que hace es **mover los atributos de un objeto a otro**, dejando el objeto que otorga los atributos en un estado vacío.

```
1 Clase(Clase&& otro) noexcept { //no except se utiliza para optimizaciones
    // del constructor
2     // Transferencia de recursos desde 'otro' a 'this'
3 }
```

- Es llamado implícitamente en los mismos casos que el constructor de copia cuando el parámetro es un valor-r. Un **valor-r (rvalue)** es un tipo de expresión que **no tiene dirección de memoria asignable**. Es decir, son valores temporales que **generalmente no pueden ser modificados directamente**. Por ejemplo:

```
1 10 = x; // ERROR: '10' es un Rvalue y no puede modificarse
2 int x = 10; // 'x' es un Lvalue
3 int y = x + 5; // 'x + 5' es un Rvalue porque es un calculo temporal
4
5 int f()
6 {
7     return 42;
8 }
9
10 int main()
11 {
12     int a = f(); // 'f()' devuelve un Rvalue
13 }
```

- El compilador genera un constructor de movimiento por defecto si el programador no define ninguno de los métodos siguientes:
 - Ctor de copia
 - Operador de asignación de copia
 - Ctor de movimiento
 - Operador de asignación de movimiento
 - Destructor
- Por lo que, debemos seguir **la regla del 5**, si **una clase maneja recursos externos**, debemos **definir todos los métodos necesarios para tratar correctamente a los objetos de la clase** (los 5 de arriba).

```

1 #include <iostream>
2 #include <cstring>
3
4 class Libro {
5 private:
6     char* titulo;
7     int paginas;
8
9 public:
10    // Constructor normal
11    Libro(const char* titulo = "", int paginas = 0)
12        : titulo(new char[strlen(titulo) + 1]), paginas(paginas) {
13        strcpy(this->titulo, titulo);
14        std::cout << "Constructor normal llamado\n";
15    }
16
17    // Constructor de copia
18    Libro(const Libro& otro)
19        : titulo(new char[strlen(otro.titulo) + 1]), paginas(otro.paginas)
20        {
21        strcpy(this->titulo, otro.titulo);
22        std::cout << "Constructor de copia llamado\n";
23    }
24
25    // Constructor de movimiento
26    Libro(Libro&& otro) noexcept
27        : titulo(otro.titulo), paginas(otro.paginas) {
28        otro.titulo = nullptr; // Se transfiere la propiedad
29        otro.paginas = 0;
30        std::cout << "Constructor de movimiento llamado\n";
31    }
32
33    // Destructor
34    ~Libro() {
35        delete[] titulo;
36    }
37
38    // Mostrar informacion
39    void mostrar() const {
40        std::cout << "Titulo: " << (titulo ? titulo : "Sin titulo")
41        << ", Paginas: " << paginas << std::endl;
42    }
43 };
44
45 int main() {
46     Libro libro1("C++ Avanzado", 500);
47
48     // Se usa el constructor de movimiento
49     Libro libro2 = std::move(libro1);
50
51     libro1.mostrar(); // Deberia mostrar un estado vacio
52     libro2.mostrar(); // Deberia mostrar los datos movidos

```

```
53 |     return 0;  
54 | }
```

6.1.7. Operador de asignación de movimiento

Definición 6.1.4

El operador de asignación de movimiento recibe un objeto de la clase (operando derecho) por referencia a valor-r para moverlo a otro preexistente(operando izquierdo) que devuelve por referencia

```
1  class C
2  {
3      public:
4          C& operator=(C&& c){...}
5  };
```

- El operador de asignación de movimiento (operator= con &&) se usa para **transferir los recursos de un objeto** a otro **sin realizar copias innecesarias**. Esto es especialmente útil **cuando la clase gestiona recursos dinámicos**, como memoria en el heap, archivos o conexiones de red.
- Es definido implícitamente por el compilador si el programador no lo define y tampoco define constructor/asignación de copia, constructor de movimiento ni destructor.
- El **operador de asignación de movimiento por defecto** asigna uno a uno los atributos de la clase, usando la asignación de movimiento del tipo de cada atributo. Los atributos de tipos básicos se copian. Por ejemplo:

```
1  #include <iostream>
2  #include <utility> // std::move
3
4  class MiClase {
5  private:
6      int* datos;
7      size_t tamano;
8
9  public:
10     // Constructor
11     MiClase(size_t tam) : tamano(tam), datos(new int[tam]) {
12         std::cout << "Constructor normal\n";
13     }
14
15     // Destructor
16     ~MiClase() {
17         delete[] datos;
18         std::cout << "Destructor\n";
19     }
20
21     // Constructor de movimiento
22     MiClase(MiClase&& otro) noexcept : datos(nullptr), tamano(0) {
23         std::cout << "Constructor de movimiento\n";
24         *this = std::move(otro);
25     }
```

```

26
27 // Operador de asignacion de movimiento
28 MiClase& operator=(MiClase&& otro) noexcept {
29     std::cout << "Operador de asignacion de movimiento\n";
30
31     // Evita autoasignacion
32     if (this != &otro) {
33         // Liberar los recursos actuales
34         delete[] datos;
35
36         // Mover recursos
37         datos = otro.datos;
38         tamano = otro.tamano;
39
40         // Dejar el objeto original en estado nulo
41         otro.datos = nullptr;
42         otro.tamano = 0;
43     }
44     return *this;
45 }
46 };
47
48 int main() {
49     MiClase a(5);           // Constructor normal
50     MiClase b(10);          // Constructor normal
51
52     b = std::move(a);        // Operador de asignacion de movimiento
53
54     return 0;
55 }

```

- **NOTA:** Debe manejar la autoasignación (if (this != &otro)).
- Debe dejar el objeto original en estado válido (otro.datos = nullptr;).
- Debe liberar los recursos anteriores antes de reasignar nuevos recursos.

6.1.8. Constructor de conversión

Definición 6.1.5

Un **constructor de conversión** recibe un parámetro de un tipo distinto al de la clase y se encarga de convertir dicho parámetro a un objeto de la clase.

```
1  class C
2  {
3      public:
4          C(const D& d){...}
5  };
```

- Es llamado cuando se realiza una conversión explícita de un valor del tipo del parámetro a un objeto de la clase o bien cuando el compilador espera encontrar un objeto de la clase y encuentra un valor del tipo del parámetro.

Constructor de conversión sin explicit

```
1  #include <iostream>
2
3  class Punto {
4  public:
5      int x, y;
6
7      // Constructor de conversión implícito
8      Punto(int valor) : x(valor), y(valor) {
9          std::cout << "Constructor de conversión implícita\n";
10     }
11 };
12
13 void imprimirPunto(Punto p) {
14     std::cout << "Punto: (" << p.x << ", " << p.y << ")\n";
15 }
16
17 int main() {
18     Punto p1 = 10; // Conversión implícita int -> Punto
19     imprimirPunto(20); // También se convierte implícitamente
20
21     return 0;
22 }
23 /* ----- SALIDA: -----
24
25 Constructor de conversión implícita
26 Constructor de conversión implícita
27 Punto: (20, 20)
28
29 */
```


Constructor de conversión con explicit

```
1 #include <iostream>
2
3 class Punto {
4 public:
5     int x, y;
6
7     // Constructor de conversión explícita
8     explicit Punto(int valor) : x(valor), y(valor) {
9         std::cout << "Constructor de conversión explícita\n";
10    }
11 };
12
13 void imprimirPunto(Punto p) {
14     std::cout << "Punto: (" << p.x << ", " << p.y << ")\n";
15 }
16
17 int main() {
18     // Punto p1 = 10; // Error: conversión implícita no permitida
19     Punto p2(10); // OK: Conversión explícita
20     imprimirPunto(Punto(20)); // OK: Conversión explícita con
        constructor
21     imprimirPunto(static_cast<Punto>(30)); // OK: Uso de static_cast
22
23     return 0;
24 }
25
26 /* ----- SALIDA: -----
27 Constructor de conversión explícita
28 Constructor de conversión explícita
29 Punto: (20, 20)
30 Constructor de conversión explícita
31 Punto: (30, 30)
32 */
```

- Sin **explicit**, la conversión ocurre automáticamente.
- Con **explicit**, la conversión requiere una llamada explícita, evitando errores accidentales. Úsalo si no quieres que la conversión suceda automáticamente, evitar errores al pasar argumentos incorrectos a funciones...

6.1.9. Operador de asignación con conversión

Definición 6.1.6

Hace exactamente lo mismo que el constructor de conversión, evitando llamar al constructor de conversión de la clase y al operador de asignación cuando se hace lo siguiente:

```
1
2  class Punto
3  {
4      public:
5          Punto& operator =(const Complejo& c) //operador de conversión
6          {x_=c.real(); y_=c.imag(); return *this;}
7          //...
8      };
9
10     //en una funcion cualquiera...
11     Complejo c1,c2;
12     Punto p1=c1,p2; //llamada al constructor de conversión
13     p2=c2; //llamada al operador de conversión
```

- **NOTA:** Si tenemos definido un constructor de conversión en dos clases, cuando trabajemos con los objetos de las dos clases podemos tener un problema de ambigüedad, donde podamos transformar en el siguiente ejemplo, de punto a complejo o de complejo a punto:

```
1  class Punto
2  {
3      public:
4          //...
5          Punto(const Complejo&C): x_{c.real()}, y_{c.imag()} {} //ctor de
6          conversión
7          Punto& operator =(const Complejo& c) //operador de conversión
8          {x_=c.real(); y_=c.imag(); return *this;}
9          //...
10     };
11
12     class Complejo
13     {
14     public:
15         Complejo(const Punto&); //ctor de conversión
```

- En el ejemplo anterior, al trabajar con puntos y complejos podemos tener un problema de ambigüedad, ya que si hacemos por ejemplo:

```

1 void f(const Complejo& z); //funcion definida en otro sitio que hara
    algo en especifico
2 Punto p;
3 f(p); //la funcion f recibe un complejo, pero le estamos pasando un
    punto. Ahora el compilador que hace, llama al metodo de la clase
    complejo para transformar el punto a complejo o llama al metodo de
    la clase punto para transformar el punto a complejo?
4 //PROBLEMA DE AMBIGUEDAD

```

- Para resolver esta ambigüedad lo que debemos de hacer es, marcar los constructores como explicit para que la conversión solo ocurra cuando nosotros se lo digamos explícitamente al compilador.
- **NOTA:** Todas las conversiones de un tipo de una clase a otra la definiremos con **el constructor de conversión**. Evitaremos el uso del operador de conversión, **excepto en los casos donde el tipo destino a convertir no sea una clase que manejemos nosotros** (por ejemplo, en las prácticas hemos de convertir una fecha a una cadena de caracteres de bajo nivel (const char*), pues debemos hacerlo con el operador de conversión). Un ejemplo completo;

```

1 #include <iostream>
2
3 class B; // Adelanto de la declaracion de B
4
5 class A {
6 public:
7     int value;
8
9     A(int v) : value(v) {}
10
11     // Constructor de conversion de B a A
12     A(const B& b);
13 };
14
15 class B {
16 public:
17     int value;
18
19     B(int v) : value(v) {}
20
21     // Constructor de conversion de A a B
22     B(const A& a) : value(a.value) {}
23 };
24
25 // Definicion del constructor de conversion de B a A
26 A::A(const B& b) : value(b.value) {}
27
28 int main() {
29     A a(10);
30     B b(20);

```

```

31
32     A result = a + b; // Error: Ambigüedad en la conversión
33     std::cout << result.value << std::endl;
34
35     return 0;
36 }
37
38 /*Aquí el problema ocurre porque cuando intentamos hacer a + b, el
39 compilador no sabe si convertir b en A o convertir a en B.*/
40
41 //SOLUCION
42 class A {
43 public:
44     int value;
45     A(int v) : value(v) {}
46
47     explicit A(const B& b);
48 };
49
50 class B {
51 public:
52     int value;
53     B(int v) : value(v) {}
54
55     explicit B(const A& a) : value(a.value) {}
56 };
57
58 A::A(const B& b) : value(b.value) {}
59
60 int main() {
61     A a(10);
62     B b(20);
63
64     A result = a + static_cast<A>(b); // Ahora la conversión es explícita
65     std::cout << result.value << std::endl;
66
67     return 0;
68 }

```

6.1.10. Constructor de Lista inicializadora (C++11)

Definición 6.1.7

Un constructor de lista inicializadora (`initializer_list`) se usa cuando queremos permitir la inicialización de un objeto con una lista de valores, similar a cómo se inicializan contenedores estándar como `std::vector` o `std::set`. (Un contenedor es una estructura de datos que almacena información como Pilas, Listas, Colas, Árboles...)

- Básicamente permite realizar creaciones de objetos otorgándole valores entre llaves
- **NOTA:** Si definimos un constructor de lista inicializadora en nuestra clase y **NO QUEREMOS QUE AL CREAR UN OBJETO SE LLAME A ESTE CONSTRUCTOR SI NO A OTRO, DEBEMOS UTILIZAR LOS PARÉNTESIS EN VEZ DE LAS LLAVES**

```
1 #include <iostream>
2 #include <initializer_list>
3
4 class MiClase {
5 public:
6     MiClase(std::initializer_list<int> lista) {
7         for (int valor : lista) { //bucle de rango
8             std::cout << valor << " ";
9         }
10        std::cout << std::endl;
11    }
12};
13
14 int main() {
15     MiClase obj = {10, 20, 30, 40}; // Se usa el constructor de lista
16                                     inicializadora
17     return 0;
18 }
19
20 /*----- SALIDA -----
21 10 20 30 40
22 */
```

- Como podemos observar en el ejemplo anterior el constructor de lista inicializadora recibe una serie de valores (enteros en este caso) y lo que hace es almacenarlos en un objeto de tipo `initializer_list` que crea automáticamente el compilador
- En este ejemplo en concreto lo único que hace es mostrar dichos elementos por pantalla, pero se podrían hacer lo que fuese oportuno con ellos. ¿Cómo se trabaja con el objeto `initializer_list`? Se verá en el siguiente punto.

Iteradores

Definición 6.1.8

Un iterador en un contenedor es un objeto que **actúa como un puntero** o un **cursor** que permite **recorrer los elementos** de dicho contenedor de manera controlada y eficiente. Son fundamentales para manipular estructuras de datos como vector, list, set, map, entre otros. Los iteradores permiten acceder a los elementos de un contenedor sin necesidad de conocer su implementación interna. Esto hace que el código sea más genérico y reusable. Por ejemplo:

```
1  #include <iostream>
2  #include <vector>
3
4  int main() {
5      std::vector<int> numeros = {10, 20, 30, 40, 50};
6
7      // Definimos un iterador para el vector
8      std::vector<int>::iterator it;
9
10     // Recorremos el vector usando el iterador
11     for (it = numeros.begin(); it != numeros.end(); ++it) {
12         std::cout << *it << " "; // Accedemos al valor con '*'
13     }
14
15     return 0;
16 }
```

- En un contenedor, la función *begin()* la podemos asociar a la función *primera()* de la lista, y la función *end()* la podemos asociar a la función fin de la lista. Esta función *end()* es un iterador (puntero al fin y al cabo) a la posición siguiente a la del último elemento.

Deducción automática de tipo (auto)

Definición 6.1.9

La palabra clave **auto** permite al compilador **deducir automáticamente el tipo de una variable** en tiempo de compilación. Esto hace que el código sea más limpio y flexible, especialmente cuando trabajamos con tipos complejos o iteradores. Un ejemplo básico:

```
1 #include <iostream>
2
3 int main() {
4     auto x = 10;           // int
5     auto y = 3.14;         // double
6     auto z = "Hola";       // const char*
7
8     std::cout << x << " " << y << " " << z << std::endl;
9     return 0;
10 }
```

- Uno de los usos más comunes de auto es simplificar la declaración de iteradores en contenedores :

```
1 #include <iostream>
2 #include <vector>
3
4 int main() {
5     std::vector<int> numeros = {10, 20, 30, 40, 50};
6
7     // Sin auto
8     std::vector<int>::iterator it = numeros.begin();
9
10    // Con auto
11    auto it_auto = numeros.begin(); // El compilador deduce std::vector<
12                                     int>::iterator
13
14    std::cout << *it_auto << std::endl; // Imprime el primer elemento
15                                     (10)
16    return 0;
17 }
18 // Con auto, evitamos escribir la larga declaracion std::vector<int>::
19 // iterator
```

- Con *auto*, lo podemos usar en un bucle for cuando vayamos a recorrer un contenedor utilizando iteradores:

```
1 #include <iostream>
2 #include <vector>
3
4 int main() {
5     std::vector<int> numeros = {10, 20, 30, 40};
6
7     // Recorrer el vector con auto
8     for (auto num : numeros) {
9         std::cout << num << " ";
10    }
11
12    return 0;
13 }
```

- Debemos tener en cuenta que *auto* nombrevariable es una copia de cada elemento del contenedor que vamos a recorrer. Para evitar hacer una copia a cada elemento, podemos usar *auto&*

```
1     #include <iostream>
2     #include <vector>
3
4     int main() {
5         std::vector<std::string> nombres = {"Ana", "Luis", "Carlos"};
6
7         for (auto& nombre : nombres) { // Usa referencia para evitar copias
8             nombre += " (editado)";    // Modificamos el original
9         }
10
11         for (const auto& nombre : nombres) { // Usa const auto& si no
12             // necesitas modificar
13             std::cout << nombre << " ";
14         }
15
16         return 0;
17     }
```


Bucles de rango

Definición 6.1.10

Los bucles de rango (range-based for loops) permiten recorrer contenedores y arreglos de una manera más sencilla y legible que los bucles for tradicionales.

```
1 for (tipo variable : contenedor) {  
2     // Cuerpo del bucle  
3 }
```

- donde:
 - **tipo**: El tipo de cada elemento en el contenedor.
 - **variable**: Nombre de la variable que representa cada elemento.
 - **contenedor**: La estructura sobre la que se va a iterar.

```
1 #include <iostream>  
2 #include <vector>  
3  
4 int main() {  
5     std::vector<int> numeros = {10, 20, 30, 40, 50};  
6  
7     // Bucle de rango  
8     for (int num : numeros) {  
9         std::cout << num << " ";  
10    }  
11  
12    return 0;  
13 }  
14  
15  
16 /*-----SALIDA----- 10 20 30 40 50  
17 */
```

- Esto evita usar índices, begin() y end().
- Por defecto, en un bucle de rango, cada elemento se copia, lo que puede ser ineficiente con estructuras grandes. Para evitar copias innecesarias, usamos referencias (&). (lo que comenté anteriormente con auto es similar a esto)

```

1 #include <iostream>
2 #include <vector>
3
4 int main() {
5     std::vector<std::string> nombres = {"Ana", "Luis", "Carlos"};
6
7     // Usamos referencia (&) para evitar copias
8     for (std::string& nombre : nombres) {
9         nombre += " (modificado)";
10    }
11
12    for (const std::string& nombre : nombres) { // Iterar sin modificar
13        std::cout << nombre << std::endl;
14    }
15
16    return 0;
17 }

```

```

1 #include <initializer_list>
2 #include <algorithm> // copy
3 class Vector {
4 public:
5     // ...
6     Vector(int n, double e = 0.0); // n eltos. de valor e
7     Vector(std::initializer_list<double>);
8     // ...
9 private:
10    double* eltos;
11    int n_eltos;
12 };
13 Vector::Vector(std::initializer_list<double> l) :
14    eltos{new double[l.size()]}, n_eltos{l.size()}
15 {
16    std::copy(l.begin(), l.end(), eltos); //equivalente a:
17
18    /*
19     for(auto&x: l)
20     {
21         *i=x;
22         ++i;
23     }
24    */
25 }
26
27 }

```

Definición 6.2.1

El destructor de una clase es una función especial que se ejecuta automáticamente cuando un objeto de esa clase es destruido (por ejemplo, cuando sale de su ámbito o se libera su memoria). Su propósito principal es liberar recursos asignados dinámicamente, cerrar archivos, desconectar conexiones, etc. Sus características son:

- Tiene el mismo nombre que la clase, pero precedido por ~
- No se puede sobrecargar.
- No tiene parámetros ni valor de retorno.
- Se ejecuta automáticamente cuando un objeto deja de existir.
- Se usa para liberar memoria y otros recursos.
- Un constructor **NO** tiene como función destruir un objeto, si no realizar **tareas previas de limpieza** que puedan ser necesarias para que sea destruido correctamente. Veamos varios ejemplos:

```

1 #include <iostream>
2
3 class Ejemplo {
4 public:
5     Ejemplo() { // Constructor
6         std::cout << "Constructor llamado" << std::endl;
7     }
8
9     ~Ejemplo() { // Destructor
10        std::cout << "Destructor llamado" << std::endl;
11    }
12 };
13
14 int main() {
15     Ejemplo obj; // Se crea el objeto, llama al constructor
16     // Cuando main() termina, el destructor se ejecuta automaticamente
17     return 0;
18 }
19
20 /*
21 ----- SALIDA ESPERADA -----
22 Constructor llamado
23 Destructor llamado
24 */

```

- Otro ejemplo con memoria dinámica:

```

1 #include <iostream>
2
3 class MiClase {
4 private:
5     int* ptr;
6 public:
7     MiClase(int valor) {
8         ptr = new int(valor); // Reserva memoria dinamicamente
9         std::cout << "Constructor: memoria asignada" << std::endl;
10    }
11
12    ~MiClase() {
13        delete ptr; // Libera la memoria
14        std::cout << "Destructor: memoria liberada" << std::endl;
15    }
16 };
17
18 int main() {
19     MiClase obj(42); // Se crea el objeto
20     return 0; // Cuando obj sale de ambito, se llama al destructor
21 }
22
23 /*
24 ----- SALIDA ESPERADA -----
25 Constructor: memoria asignada
26 Destructor: memoria liberada
27 */

```

6.2.1. Consideraciones

- Si una clase tiene miembros que usan **memoria dinámica**, se debe implementar un **destructor para liberar esa memoria**.
- Si **no se define un destructor**, el compilador genera **uno por defecto** (pero no libera memoria dinámica).
- Si una clase tiene **herencia con punteros**, se recomienda definir el **destructor como virtual** en la clase base para evitar fugas de memoria. (se verá más adelante)

MÉTODOS ESPECIALES

Definición 7.0.1

Son aquellos métodos para los que, bajo ciertas condiciones, se generan automáticamente por el compilador

- El programador puede obviar estas definiciones por defecto utilizando **default** (utiliza el método por defecto del compilador) o **delete** (suprime el método por defecto del compilador)

```
1  class C {  
2  public:  
3  C() = default;  
4  C(const C&) = delete; // Los objetos de C  
5  C& operator =(const C&) = delete; // no se pueden copiar,  
6  C(C&&) = default; // pero si  
7  C& operator =(C&&) = default; // mover  
8  ~C() = default;  
9  // ...  
10 }
```